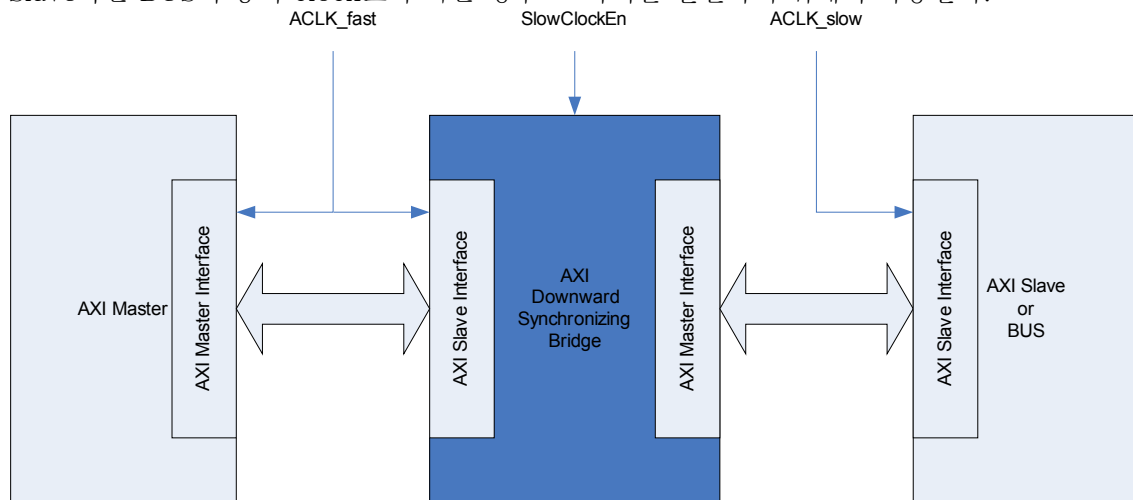


Title	AXI downward synchronizing Bridge Integration Guide
Project	None
Description	AXI downward synchronizing Bridge Integration Guide
Author	holelee
Date	24/May/2005

(*) AXI Downward Synchronizing Bridge의 사용

AXI Downward Synchronizing Bridge(이하 DS Bridge)는 AXI Master의 동작 clock이 AXI Slave혹은 BUS의 동작 clock보다 빠른 경우 그 사이를 연결하기 위해서 사용한다.



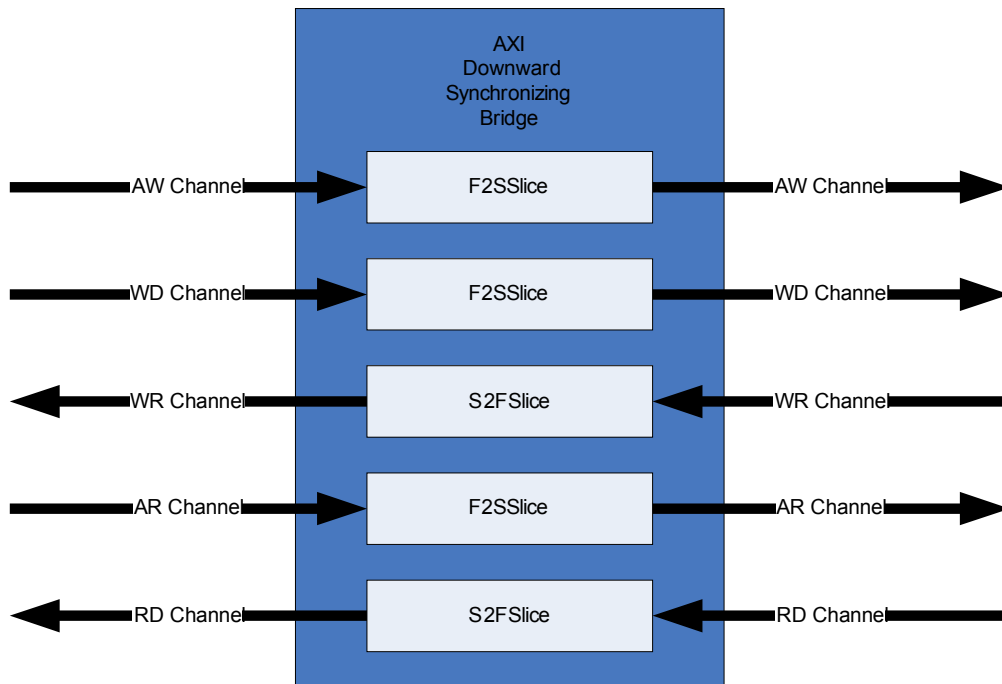
단, 언제나 사용할 수 있는 것은 아니고 AXI Master의 동작 clock이 AXI slave의 동작 clock의 정수배이어야 하는 조건과 Slave의 clock의 rising edge가 Master clock과 동기되어 있다는 조건을 만족시킬 경우 사용할 수 있다.

기본적으로 DS Bridge는 빨리 동작하는 AXI Master를 느린 AXI BUS에 연결하는 것을 감안하고 설계되었다. 따라서 Master쪽(Fast side)의 버스 점유 cycle은 크게 중요하지 않고 Slave쪽(Slow side)의 버스 bandwidth를 최대한 활용하는 구조로 되어있다. Slow side에서 전달되는 정보를 buffering하였다가 Fast side를 최저의 cycle만 사용하여 전송하는 구조가 아니다. 따라서 AXI master쪽이 AXI BUS이고 Slave쪽이 느리게 동작하는 AXI Slave라고 하면 느리게 동작하는 Slave 속도에 맞추어 정보(Read data가 가장 문제가 됨)가 전달되게 되므로 빠른 AXI BUS의 bandwidth에 문제가 생길 수 있다.

(*) DS Bridge의 형태

AXI interface는 5개의 독립된 channel을 가지고 있으므로 모든 채널에 대해서 따로 만들 필요는 없다. Master에서 Slave로 정보가 전달되는 채널(Write Address, Write Data, Read Address)과 Slave에서 Master로 정보가 전달되는 채널(Write Response, Read Data)로 구분하여 두 가지를 만들어 5개의 채널에 instantiation하여 사용하도록 했다. 두 가지는 각각 S2FSlice(Slow to Fast Slice)와 F2SSlice(Fast to Slow Slice)인데 이것을 다음 그림과 같이

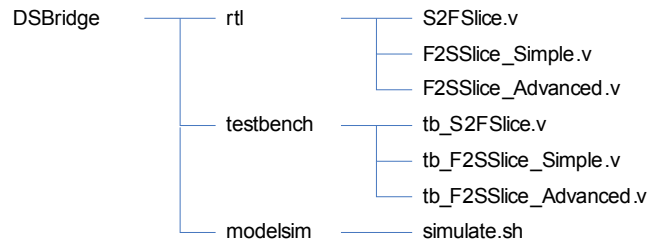
instantiation하여 연결하면 전체 AXI Downward Synchronizing Bridge가 완성된다. 채널별로 전달되는 정보(address, id, data 등)의 bit-width가 다르므로 그 bit-width는 verilog parameter를 통해서 configuration가능하도록 만들어서 instantiation을 할때 넘겨주도록 하였다.



S2FSlice는 완전한 timing isolation이 된다. 즉, S2FSlice 사이로 전달되는 모든 signal은 register로 latching된 후에 전달된다. F2SSlice는 두가지 버전이 있는데 하나는 Simple이고 다른 하나는 Advanced로 각각 F2SSlice_Simple과 F2SSlice_Advanced로 구분했다. 두 경우 모두 Valid/Information은 register로 latch된 상태에서 전달지만, Simple의 경우 Slave에서 오는 READY 신호가 combinational path를 통해서 Master쪽으로 전달되는 반면 Advanced의 경우는 READY 신호도 register로 latch되어 전달된다. 두 clock 사이에 완전한 timing isolation을 하고 싶다면 Advanced를 사용하면 된다. 이러한 Simple, Advanced는 각각 AXI Register Slice의 registered forward path, fully registered의 경우와 비슷하다고 볼 수 있다.

(*) 제공되는 module 및 파일

제공 되는 verilog source code는 S2FSlice, F2SSlice_Simple, F2SSlice_Advanced에 대해 각각 S2FSlice.v, F2SSlice_Simple.v, F2SSlice_Advanced.v 이다. 각각의 module을 테스트 하기 위한 test bench로 tb_S2FSlice.v, tb_F2SSlice_Simple.v, tb_F2SSlice_Advanced.v가 제공되고, compile과 simulation을 위해 simulate.sh이 존재한다.



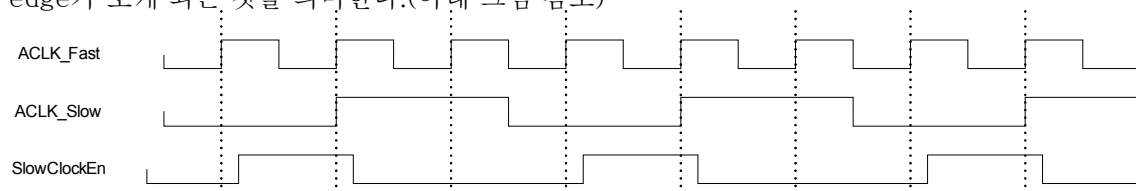
Test bench는 random data를 공급한 후 Bridge를 통과한 data를 원래 data의 값과 비교하는 식으로 동작하도록 했다. simulation을 하기 위해서는 simulate.sh를 수행시키면 된다.

(*) Signals

- S2FSlice

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
ACLK_Fast	IN	Clock Generator	AXI clock(Fast side)
ARESETn	IN	Reset Controller	AXI reset
SlowClockEn	IN	Clock Generator	Slow Clock Rising Indicating Signal
INFORMATION_F[X:0]	OUT	Master(Fast)	Information such as addr/data/control
VALID_F	OUT	Master(Fast)	VALID signal to Fast-side
READY_F	IN	Master(Fast)	READY signal from Fast-side
INFORMATION_S[X:0]	IN	Slave(Slow)	Information such as addr/data/control
VALID_S	IN	Slave(Slow)	VALID signal from Slow-side
READY_S	OUT	Slave(Slow)	READY signal to Slow-side

ACLK_Fast는 fast-side, 즉 master가 사용하는 AXI clock을 연결해야 한다. SlowClockEn은 slow-side, 즉 slave가 사용하는 AXI clock의 rising edge를 detect하는 signal로 SlowClockEn이 high이면 다음 ACLK_Fast의 rising edge에 slow-side AXI clock의 rising edge가 오게 되는 것을 의미한다.(아래 그림 참조)



만약 SlowClockEn이 항상 high라면 그것은 곧 ACLK_Slow와 ACLK_Fast가 동일한 clock임을 의미하며 그럴 경우 S2FSlice는 AXI Register Slice와 동일한 behavior를 가지게 된다. 하지만 Register Slice에 비해 gate count가 클 가능성이 있으므로 Register Slice를 사용할 곳에 S2FSlice를 사용하지 않는 것을 권장한다.

INFORMATION_F와 INFORMATION_S의 bit-width는 verilog parameter WIDTH로 결정되므로 사용하는 channel 사이에 전달되는 정보의 양에 따라서 configuration하여 사용하면 된다.

- F2SSlice_Simple/F2SSlice_Advanced 공통

F2SSlice_Simple과 F2SSlice_Advanced는 다음과 동일한 입출력 포트를 가지고 있다. 각 signal이 가지는 의미는 S2FSlice와 동일하고 입출력 방향만 바뀐 것이다.

<i>signal</i>	<i>방향</i>	<i>Source/ Destination</i>	<i>Description</i>
ACLK_Fast	IN	Clock Generator	AXI clock(Fast side)
ARESETn	IN	Reset Controller	AXI reset
SlowClockEn	IN	Clock Generator	Slow Clock Rising Indicating Signal
INFORMATION_F[X:0]	IN	Master(Fast)	Information such as addr/data/control
VALID_F	IN	Master(Fast)	VALID signal from Fast-side
READY_F	OUT	Master(Fast)	READY signal to Fast-side
INFORMATION_S[X:0]	OUT	Slave(Slow)	Information such as addr/data/control
VALID_S	OUT	Slave(Slow)	VALID signal to Slow-side
READY_S	IN	Slave(Slow)	READY signal from Slow-side

(*) Integration 방법

Integration 방법은 AXI Register Slice의 integration 방법과 동일하다. Read Data channel에 대한 예를 살펴보면 다음과 같다. INFORMATION에는 RID,RDATA,RRESP,RLAST를 concatenation을 하여 연결하고, 그 bit-width에 맞추어 width parameter를 주면 된다.

```

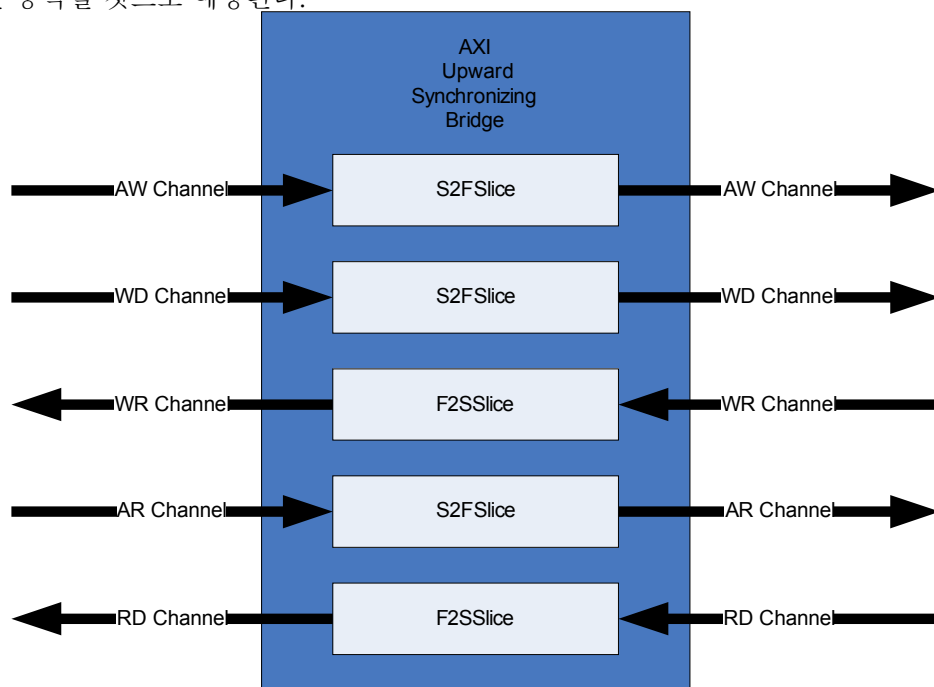
S2FSlice #(width) rd_s2fslice(
    .ACLK_Fast(ACLK_Fast),
    .ARESETn(ARESETn),
    .SlowClockEn(SlowClockEn),
    .INFORMATION_F({RID_Master,      RDATA_Master,      RRESP_Master,
RLAST_Master}),
    .VALID_F(RVALID_Master),
    .READY_F(RREADY_Master),
    .INFORMATION_S({RID_Slave,      RDATA_Slave,      RRESP_Slave,
RLAST_Slave}),
    .VALID_S(RVALID_Slave),

```

```
.READY_S (RREADY_Slave)  
);
```

(*) 참고 : AXI Upward Synchronizing Bridge

S2FSlice와 F2SSlice를 이용하여 AXI Upward Synchronizing Bridge도 다음과 같이 만들 수 있고, 잘 동작할 것으로 예상된다.



다만 이런 경우에는 AXI Master쪽(Slow-side)이 AXI BUS이고, AXI Slave쪽(Fast-side)가 AXI Slave인 경우에는 BUS bandwidth를 최대로 활용할 수 있다. 반대의 경우에는 BUS를 통해 정보가 전달되는 cycle수가 느린 쪽에 따라서 늘어나게 되는 단점이 생겨, BUS의 bandwidth를 효율적으로 사용하지 못할 가능성이 있다. 이 문제를 해결하기 위해서는 slow-side에서 fast-side로 전달되는 부분(특히 Write Data channel)에 buffer를 넣어 buffering이 완료되면 한꺼번에 fast-side쪽으로 보내는 식으로 해결해야 한다.